MATHEMATICAL DERIVATIONS

1. Notation and Shapes

Let a batch contain B examples(batch size). Each example has L = 24 hourly observations (sequence length). The model representation size is d_model = 4. The attention key/query dimension is d_k = 4 and the value dimension is d_v = 4.

$$X \in R^{(B \times L \times d_model)}, \quad L = 24, \quad d_model = d_k = d_v = 4.$$

For one sequence, X contains 24 vectors, one for each observed hour. Self-attention compares every position with every other position, so the attention matrix contains one score for every ordered pair of positions.

$$Q, K \in R^{(B \times L \times d_k)}, \quad V \in R^{(B \times L \times d_v)}.$$

$$Scores \in R^{(B \times L \times L)}, \quad AttentionWeights \in R^{(B \times L \times L)}.$$

2. From Scalar Demand to a Model Representation

Each hourly demand value is initially a scalar. The forecasting model maps it to the four-dimensional representation space using a learned linear projection.

$$h_t = x_t W_E + b_E, \quad x_t \in R, \quad h_t \in R^4.$$

The purpose of this projection is not to change the forecasting target. It provides a small learned feature space in which query, key, and value projections can compare temporal observations using multiple learned dimensions.

3. Positional Encoding

Self-attention is permutation-equivariant if position information is not supplied: the same set of feature vectors could be rearranged without giving the mechanism an explicit notion of order. The forecasting problem is inherently ordered, so deterministic sinusoidal position features are added.

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d_model)})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d_model)})$$

For the 24 positions, the model forms:

$$X_pos = H + PE.$$

The different frequencies give the model position-dependent patterns without introducing additional learned position parameters. In this project, the position encoding is deterministic and registered as a model buffer, so it is not optimized during training.

4. Query, Key, and Value Projections

The implemented attention layer learns three parameter matrices. Because this is self-attention, all three are computed from the same input representation X_pos.

$$Q = X_{pos} W_Q$$

$$K = X_{pos} W_K$$

$$V = X_{pos} W_V$$

Here $W_Q \in R^{(d_{model} \times d_k)}$, $W_K \in R^{(d_{model} \times d_k)}$, and $W_V \in R^{(d_{model} \times d_v)}$. With $d_{model} = d_k = d_v = 4$, all three matrices are 4×4 in the warehouse model.

Interpretation: a query is what the current position is looking for; a key describes what each position offers for matching; a value contains the information that will actually be aggregated. The three projections are separate because matching and content aggregation do not need to use the same learned coordinates.

5. Similarity Scores and the Scaling Term

$$S = Q K^T.$$

For one sequence, S_{ij} is the dot product between the query at position i and the key at position j. A large positive value means the two projected vectors are aligned in the learned representation space.

$$S_{scaled} = S / \sqrt{d_k}.$$

Why divide by $\sqrt{d_k}$? Under the simplified assumption that the query and key components are approximately independent, zero-mean, unit-variance variables, the dot product is the sum of d_k products. Its variance therefore grows roughly with d_k, so its standard deviation grows roughly as $\sqrt{d_k}$. Dividing by $\sqrt{d_k}$ keeps the score scale more stable as the dimension changes.

This matters because the next operation is an exponential softmax. Very large magnitudes can make the softmax extremely peaked and can create poor numerical conditioning. The scaling term therefore supports both stable optimization and a more controlled score distribution.

6. Row-Wise Softmax

For each query position i, the scores across all key positions are converted into non-negative weights that sum to one.

$$A_{ij} = \exp(S_{ij}) / \Sigma_j \exp(S_{ij}).$$

$$\Sigma_j A_{ij} = 1, \quad A_{ij} \geq 0.$$

Each row is therefore a probability-like distribution over the 24 input positions. This is why attention weights can be interpreted as a learned allocation of influence, although they should not be treated as a complete causal explanation of the model.

7. Numerically Stable Softmax

The implementation does not exponentiate the raw scores directly. For each row it subtracts the maximum score m before applying exp.

$$m = \max_j S_{ij}$$

$$A_{ij} = \exp(S_{ij} - m) / \Sigma_j \exp(S_{ij} - m).$$

The transformation is mathematically identical because the same constant is subtracted from every term in the row:

$$\exp(s_i-c) / \Sigma_j \exp(s_j-c) = \exp(s_i) / \Sigma_j \exp(s_j).$$

Numerically, however, exp(s_i - m) has no positive exponent larger than zero. This prevents overflow for large positive scores. The implementation was explicitly tested using scores around 1000; the stable weights remained finite and each row still summed to one.

8. Weighted Value Aggregation

$$O = A V.$$

The output at position i is the weighted sum of all value vectors:

$$O_i = \Sigma A_{ij} V_j.$$

This is the central information-mixing step. If a position assigns large weight to a particular earlier hour, that hour contributes more strongly to its output representation. Because the weights are learned from Q and K, the aggregation can change with the input sequence.

9. Forecasting Head

The warehouse model uses the attention output at the final observed position, corresponding to the most recent hour in the 24-hour input.

$$z = O_L, \quad z \in R^4.$$

$$\hat{y}_{scaled} = z W_O + b_O.$$

The scalar output is trained on standardized demand. At prediction time it is mapped back to the original demand scale:

$$\hat{y} = \hat{y}_{scaled} \sigma_{train} + \mu_{train}.$$

The reason for using the final position is causal alignment: that representation summarizes information from the entire observed context while corresponding to the forecasting time immediately before the unknown next hour.

10. Training Objective

$$L = (1/N) \Sigma_i (\hat{y}_i - y_i)^2.$$

This is mean squared error (MSE). Differentiating one example with respect to the scalar prediction gives:

$$\partial L/\partial \hat{y}_i = 2(\hat{y}_i - y_i) / N.$$

The gradient is proportional to the prediction error. Large misses therefore generate larger updates, which matches the project goal of making severe forecast errors visible in the training objective and later in RMSE.

11. Backward Derivation Through Attention

Let G_O = ∂L/∂O be the upstream gradient arriving from the forecasting head. The attention backward pass can be derived in reverse order.

11.1 Gradient with respect to V

$$O = A\,V \;\Rightarrow\; G_V = A^{T}\,G_O.$$

Each value vector contributes to every output position according to the attention weight assigned to it.

11.2 Gradient with respect to A

$$G_A = G_O\,V^{T}.$$

An attention weight receives gradient from the value vector it selects and from the upstream error at the corresponding output position.

11.3 Gradient through row-wise softmax

For one row, let a = softmax(s). The softmax Jacobian is:

$$\partial a/\partial s = \mathrm{diag}(a) - a\,a^{T}.$$

For a row gradient g_a = ∂L/∂a, the equivalent elementwise expression is:

$$g_s = a \odot [g_a - (g_a \cdot a)\,1].$$

Applying this independently to all rows yields the gradient G_S with respect to the score matrix.

11.4 Gradient through scaled dot products

$$S = (Q\,K^{T}) / \mathrm{sqrt}(d_k).$$

$$G_Q = G_S\,K / \mathrm{sqrt}(d_k).$$

$$G_K = G_S^{T}\,Q / \mathrm{sqrt}(d_k).$$

The same scaling factor appears in both derivatives because it multiplies the forward score expression.

11.5 Gradients of the learned projections

$$Q = X\,W_Q \;\Rightarrow\; G_WQ = X^{T}\,G_Q.$$

$$K = X\,W_K \;\Rightarrow\; G_WK = X^{T}\,G_K.$$

$$V = X\,W_V \;\Rightarrow\; G_WV = X^{T}\,G_V.$$

For batched tensors, these products are summed over the batch dimension in the ordinary way. These are the parameters directly checked by the finite-difference experiment.

12. Finite-Difference Gradient Verification

Automatic differentiation is useful, but it does not independently prove that the forward equations were implemented correctly. The project therefore computes a numerical derivative for every scalar in W_Q, W_K, and W_V and compares it with PyTorch autograd.

$$g_i^{FD} = [L(\theta_i + \varepsilon) - L(\theta_i - \varepsilon)] / (2\varepsilon).$$

Central difference is used because the leading first-order truncation errors from the positive and negative perturbations cancel, making it more accurate than a one-sided difference at the same step size.

The check uses float64 and ε = 1e-5. Float64 reduces round-off error in the numerical derivative. The loss is mean squared error between the attention output and a deterministic random target. Each parameter value is perturbed, the loss is evaluated, and the original value is restored.

$$\text{Relative error} = \|g_{autograd} - g_{FD}\| / (\|g_{autograd}\| + \|g_{FD}\|).$$

Observed verification results from the implemented check:

$$W_Q: \text{relative error} = 8.221 \times 10^{-11}$$

$$W_K: \text{relative error} = 5.175 \times 10^{-11}$$

$$W_V: \text{relative error} = 2.037 \times 10^{-11}$$

These values are extremely small and show close agreement between the independently calculated numerical derivatives and autograd derivatives for all three attention projection matrices.

13. Why the Gradient Check Is Relevant to This Model

The warehouse model depends on gradients passing through several coupled operations: projection, positional addition, query/key/value transformations, score scaling, softmax, weighted aggregation, and the output head. An implementation error in the attention equations could still produce finite tensors and a superficially trainable model. The finite-difference check therefore tests the local derivative behavior independently of PyTorch autograd.

The test does not prove that the learned model is statistically optimal. It establishes a narrower claim: for the tested numerical setup, the gradients of the implemented attention parameters agree with an independent central-difference approximation.

14. Toy Learning as a Mathematical Sanity Check

Before warehouse forecasting, the same attention implementation is trained on a controlled next-value problem with eight observations. The target is a known weighted combination of the four most recent inputs:

$$y = 0.60\,x[-1] + 0.25\,x[-2] + 0.10\,x[-3] + 0.05\,x[-4].$$

The experiment compares scaled attention, S/sqrt(d_k), against unscaled attention, S. Both models use the same data, architecture, optimizer, learning rate, number of epochs, and initialization seed; only score scaling changes.

In the recorded run, both variants learned substantially: the scaled model decreased from MSE 1.977446 at epoch 1 to 0.088134 at epoch 100, while the unscaled model decreased from 2.267351 to 0.087712. The final losses are very close, so this run should be interpreted as evidence that both variants can learn the controlled task, while score scaling changes the optimization trajectory rather than guaranteeing a lower final loss in this small experiment.

15. End-to-End Mathematical Chain

24 raw demands → linear projection → positional encoding → Q,K,V → scaled QK^T → stable softmax → AV → final-position vector → linear head → standardized forecast → inverse standardization.

The chain matters because each stage has a distinct mathematical role. Projection creates a learned feature space; positional encoding supplies order; Q and K determine relevance; V carries information; scaling controls score magnitude; softmax turns scores into normalized weights; AV mixes temporal information; the final linear head maps the representation to one forecast value; and inverse standardization returns the prediction to warehouse-demand units.

Conclusion. The mathematical construction used in the project is explicit from input sequence to prediction and from loss back to the attention parameters. The independent gradient check and numerical-stability test verify two implementation properties that are not established by a normal training run alone.